THRIVE CONFERENCE
Organized by Xnet your reliable IT partner
9th - 10th June 2026
THRIVECONF.COM #ThriveITConf

Christian Nagel
Microsoft MVP

How Copilot can help development with Visual Studio 2026

THRIVE CONFERENCE

We would like to thank our sponsors

LightningTools

Pearson

span

Leading Learning Partners Association

FALA VSE RASTE!

DoNatural Cockta Argeta barcaffè

Conditus moji sladki svet

Lisjak 1950

Fino s Kristino

Hydro Flask We Make It. You Own It.

Računalniške novice

IVARNOST.SI INFORMACIJSKA VARNOST


Visual Studio
AI-assisted development
Build more. Worry less.
Microsoft
File Edit View Git Project Build Debug
ChatViewModel.cs
NotificationService.cs
namespace ContosoChat.ViewModels;
public class ChatViewModel : ObservableObject
{
private readonly IChatService _chatService;
public ObservableCollection<ChatMessage> Messages { get; } = new();
public ChatViewModel(IChatService chatService)
{
_chatService = chatService;
LoadMessages ();
}
private async void SendMessage(string message)
{
if (string.IsNullOrWhiteSpace(message))
{
return;
},
var sentMessage = new ChatMessage
{
Content = message,
Author = "User",
Timestamp = DateTimeOffset.Now
};
},
GitHub Copilot
Preview
Here's a suggested implementation of SendMessageAsync.
public async Task SendMessageAsync(string
if (string.IsNullOrWhiteSpace(de
return;
}
var sentMessage = new ChatMessage
Content = message,
Author = "User",
Timestamp = DateTimeOffset.Now
};
Messages.Add(sentMessage);
await _chatService.SendMessageAsyn
Accept
Ask a question or type '/' for commands
0/2000
Solution Explorer
Git Changes
Ready
Search

Christian Nagel

- 20+ years of experience with C# and .NET
- Author of 20+ books on C# and .NET (latest: "Modern C# in .NET 10")
- Microsoft MVP for C# and .NET
- Speaker at global conferences (TechEd, BASTA!, Thrive… for 20+ years)
- Passionate about language design and developer productivity

Topics

1. Foundations - guarantees, pricing, models
2. AI ask or agent - what's offered by a LLM?
3. GitHub Copilot Chat - how to use it in Visual Studio 2026
4. Using AI features from the normal flow
5. Using skills

Tools

- GitHub Copilot – using Copilot directly from the portal
- Copilot CLI
- GitHub Copilot App
- Visual Studio Code
- Visual Studio 2026

1) Foundations

PROTECTION
GUARANTEES

We are committed to protecting your data and upholding your rights with the highest legal and security standards.

LEGAL COMPLIANCE

We comply with applicable laws and regulations, including GDPR and other data protection frameworks.

LAWFUL & TRANSPARENT PROCESSING

Your data is processed lawfully, fairly and transparently for clear and legitimate purposes only.

DATA SECURITY

We implement industry-leading technical and organizational measures to protect your data against unauthorized access, loss or misuse.

YOUR RIGHTS

You have the right to access, rectify, erase, restrict or object to the processing of your data in accordance with applicable law.

DATA PROTECTION BY DESIGN

Privacy and data protection are built into our processes and systems from the ground up. We collect only what is necessary and minimize data wherever possible.

TRUST
Your trust is our

ACCOUNTABILITY
We are accountable for

TRANSPARENCY
Clear information,

RESPONSIBILITY
We respect your privacy

IP Ownership

- GitHub Copilot is trained on public code and licensed to users with IP guarantees
- You own your code and Copilot suggestions
- Indemnification (Defense of Third-Party Claims)
  - enable Duplicate Detection = Block
- Microsoft and GitHub have legal commitments to protect user IP and privacy
- For more details GitHub Copilot documentation and Microsoft Dev Blogs

Data Protection Guarantees

- GitHub Business and Enterprise
  - Copilot does not retain or use your code for training
- GitHub Free, Pro, Pro+
  - Copilot may use your code for training
  - You can opt-out of training data sharing in settings

Pricing

Pricing is usage-aware, and model selection affects cost.

- **Before (Premium Requests):** one request could span ~60 minutes processing time on the backend, so a single request was a premium action. Usage changed.
- **Now (AI Credits):** based on tokens processed, with different models consuming different amounts of credits per request.
- **Impact for teams:** selecting the model is a quality/performance decision and a cost decision.

Pricing Practical View

- Base license still defines who can use Copilot features.
- Model calls are measured with a relative consumption unit.
- Higher-capability models consume more units per request than faster baseline models.
- Admin/billing views are needed to track org-wide usage and optimize defaults.

Models (1)

Model choice affects speed, reasoning depth, and cost profile.

Fast general-purpose models

- Best for: chat, quick code suggestions, drafting docs
- Examples: **GPT-4.1**, **GPT-5 mini**, **Gemini Flash**
- Tradeoff: lower deep-reasoning quality on complex tasks

Reasoning-focused models

- Best for: architecture tradeoffs, debugging complex issues, multi-step planning
- Examples: **o3**, **GPT-5 reasoning**, **Claude Opus**
- Tradeoff: typically slower and higher consumption per request

Models (2)

Multimodal-capable models

- Best for: image/file understanding in addition to text
- Examples: **GPT-4.1**, **GPT-4o**, **Gemini 2.5 Pro**
- Tradeoff: use when multimodal context is needed, usually with higher consumption

Models (3)

How they differ in practice:

- Response latency (fast vs deep)
- Quality on complex reasoning
- Context/tool behavior depending on the model profile
- Throughput and consumption/cost characteristics

Important for enterprises:

- Available models are controlled by your **organization policies**.
- Admins must **enable approved models** for users/teams before they appear in the selector.
- Not every tenant gets every model at the same time (rollout, licensing, region, compliance settings).

What can an LLM do?

- Generate code snippets, explanations, and documentation

Demo: ask for the current time

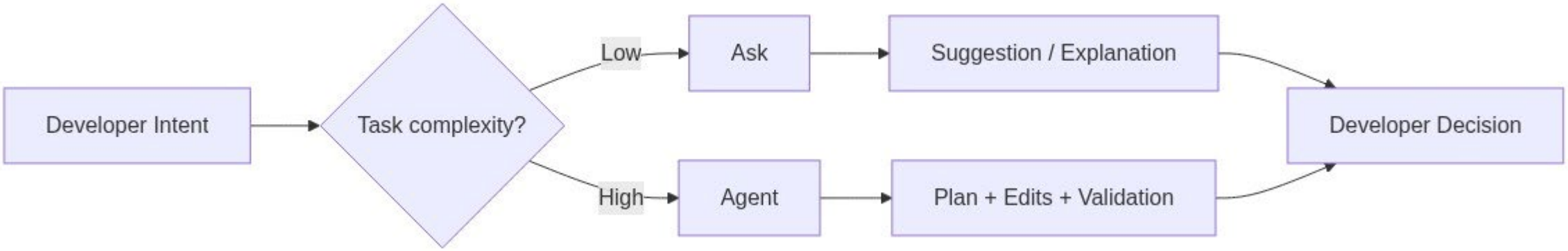
Tools

- Primarily for agent mode
- Execute code, access files, run diagnostics
- Example: time server tool for current time queries
- MCP servers accessible locally or via HTTP JSON
- Built-in tools
- MCP tools
- Planning tools
- Tool approvals and enterprise allowlists help control data access

2) AI ask or agent - what's offered by an LLM?

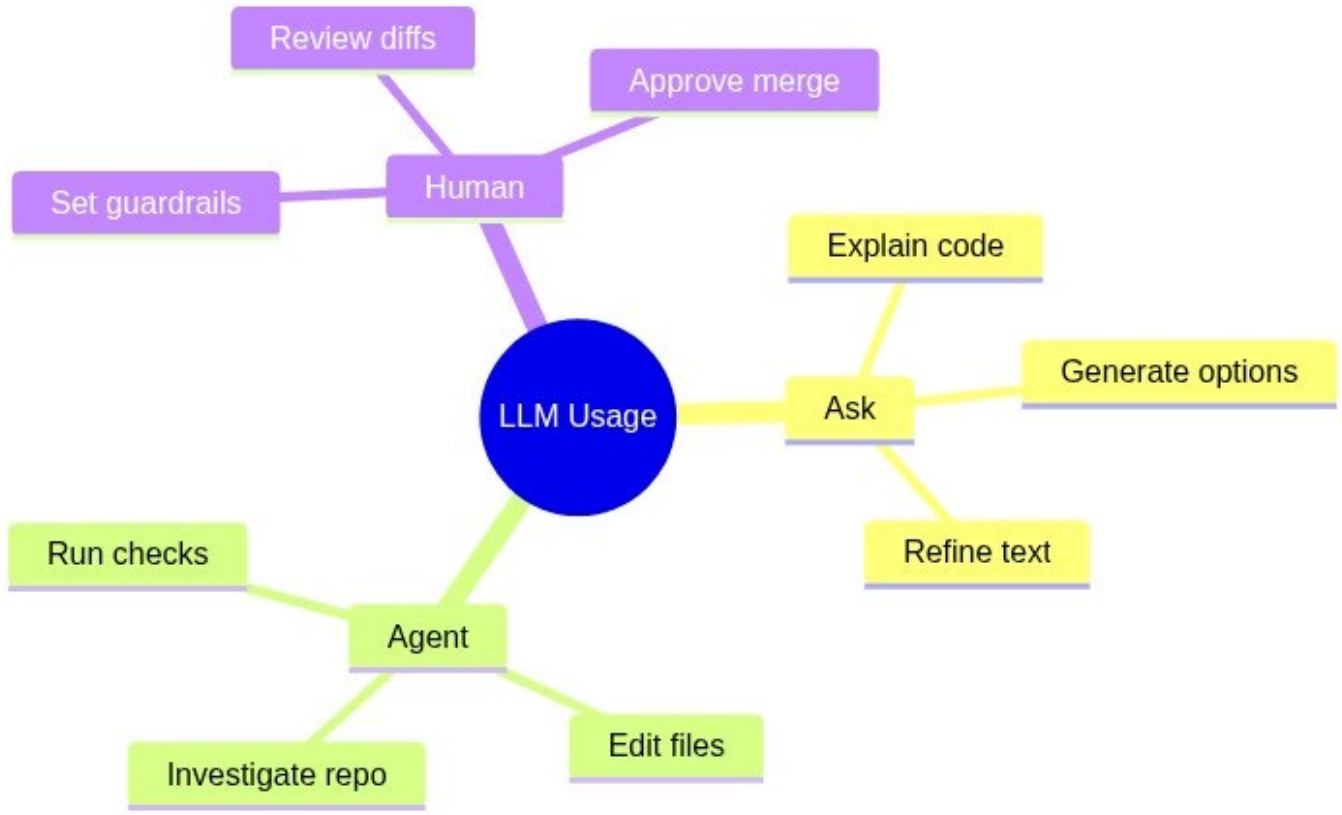
Two interaction modes, different strengths:

- **Ask mode**: quick answers, explanations, short edits
- **Agent mode**: multi-step execution, file changes, validation
- **Best results**: clear goal + constraints + expected output



Ask vs Agent decision guide

- Choose **Ask** for understanding, brainstorming, and focused snippets
- Choose **Agent** for cross-file changes and repeatable workflows
- Keep human review in the loop for architecture and tradeoffs



Plan mode (before agent execution)

- Use **Plan** mode when you want a reviewable implementation plan before code changes.
- Best for multi-file changes, unclear scope, or risky refactoring.
- Review/adjust the plan first, then execute with Agent mode.
- This improves alignment, reduces rework, and lowers token waste from retries.

Create a plan with the Codebreaker.GameAPIs project to add an API with ranking information about the players. This service should offer functionality interesting for players.

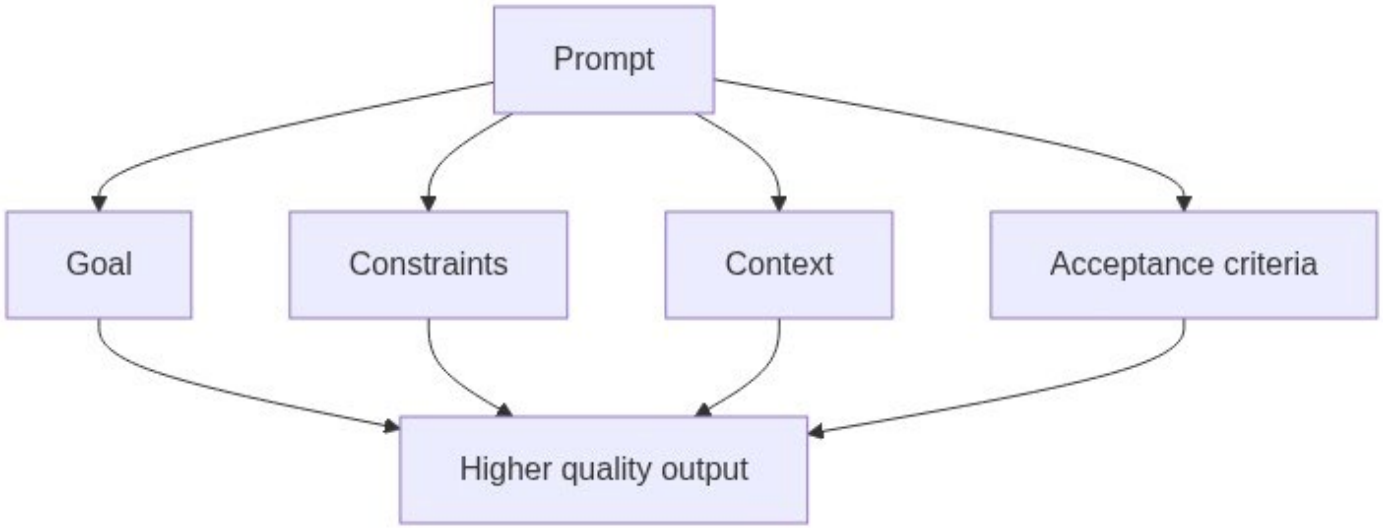
3) GitHub Copilot Chat in Visual Studio 2026

Copilot Chat is integrated into daily coding flow.

- Chat on selected code, file, or solution context
- Ask for fixes, refactors, tests, docs
- Iterate with follow-up prompts instead of restarting

Prompt patterns that work

- **Goal**: “Optimize this query method for readability.”
- **Constraints**: “Keep public API unchanged.”
- **Quality bar**: “Add tests for edge cases.”
- **Context**: “Follow patterns from ServiceX and RepositoryY.”



Context

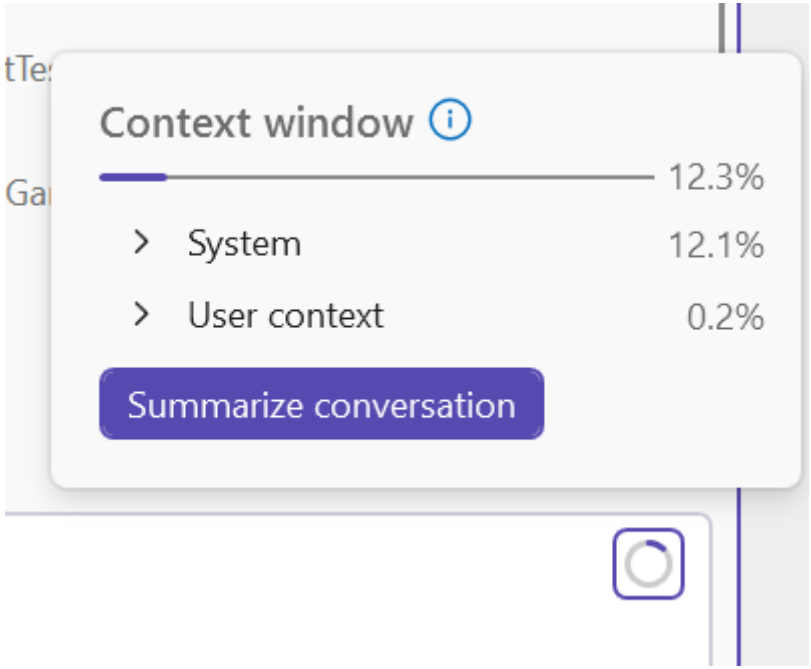
Copilot quality depends heavily on the context it can access.

- **Active document**: the file you are currently editing has highest relevance.
- **Selected code**: selected text strongly steers the response and edits.
- **Open files/tabs**: related open documents are often considered for nearby patterns.
- **Solution/workspace context**: project structure, symbols, and referenced files can be used (especially in Agent mode).
- **Explicit references in chat**: add files, PRs, issues, or MCP resources to ground answers.

Practical tip: if output is too generic, narrow scope to selected code and add explicit file references.

Chat history and context window

- Chat history is part of the prompt context for the current thread.
- Start a **new thread** for a new task/topic to avoid context bleed.
- If you want to stay in one thread, use **Summarize conversation** (or /compact) when context usage gets high.
- Keep one thread per task for better relevance and more predictable results.



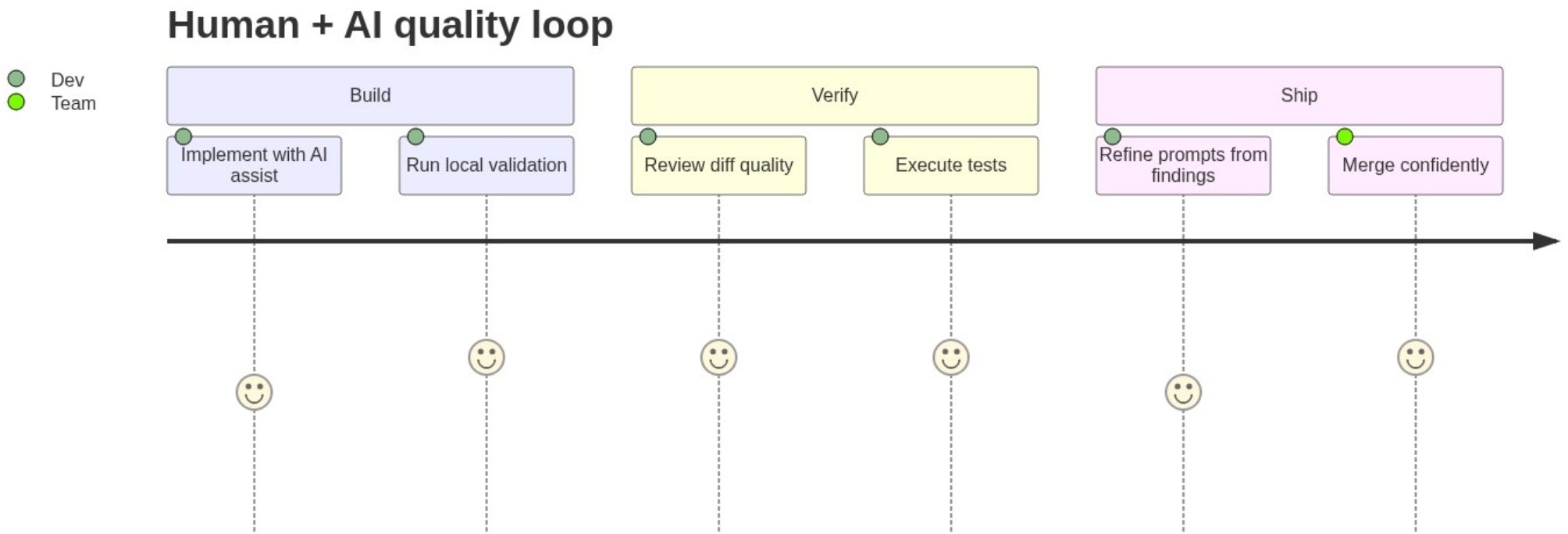
4) Using AI features from the normal dev flow

AI should be available where you already work:

- Inline completions during typing
- Quick actions and code fixes
- Add C# XML documentation for methods and classes
- Test scaffolding from existing methods
- Explain/build error fixes from Error List
- Commit message drafting from staged changes
- Pull request title/description drafting
- PR summary and review comment suggestions

Keep quality high with AI in the loop

- Treat suggestions as **proposals**, not truth
- Validate behavior with tests and diagnostics
- Prefer small, reviewable increments



When not to use AI

- **Security-critical or compliance-critical decisions** without human review.
- **High-risk production changes** that are not backed by tests/rollout safeguards.
- **Tasks requiring authoritative legal, contractual, or policy interpretation**.
- **Unknown or low-quality context** where Copilot would likely guess.
- **Rule of thumb:** use AI to accelerate execution, not to replace accountability.

5) Using skills

A skill is a reusable workflow package for a specific task domain.

- A skill usually contains: scoped instructions, recommended steps/checklists, and guardrails.
- A skill can reference or orchestrate **tools** (built-in and MCP tools), but a skill itself is not an MCP server.
- Think of it as: **process + best practices + tool usage guidance** for repeatable outcomes.
- Use skills for specialized workflows
- Reduce prompt repetition across sessions
- Improve consistency of outcomes

Copilot Memory (new) (1)

- Copilot can learn recurring preferences from your interactions (for example: coding style, naming, testing expectations).
- Memories help reduce repeated prompting and improve consistency across sessions.
- You stay in control: review, edit, or remove saved memories in settings.
- Best use: stable team conventions and personal defaults, not task-specific one-off instructions.
- Stored with github.com | Copilot | Memory

Copilot Memory (new) (2)

How this relates to skills and instructions:

- **Memory** captures evolving preferences from usage.
- **copilot-instructions.md** defines explicit repository rules.
- **Skills/agents** define task workflows.
- Use all three together for best consistency and lower prompt overhead.

Repository guidance files for Copilot (1)

- **.github/copilot-instructions.md**: default coding instructions for the whole repository.
  - Put coding conventions, architecture rules, naming/style, test expectations, and security constraints here.
- **.github/prompts/*.prompt.md**: reusable prompt templates for recurring tasks (for example: API endpoint, migration, test generation).
- **.github/agents/*.agent.md**: custom agent definitions for specialized workflows.
  - Use these when you want role-based behavior (for example: reviewer, performance specialist, modernization agent).

Repository guidance files for Copilot (2)

Why this also helps token usage:

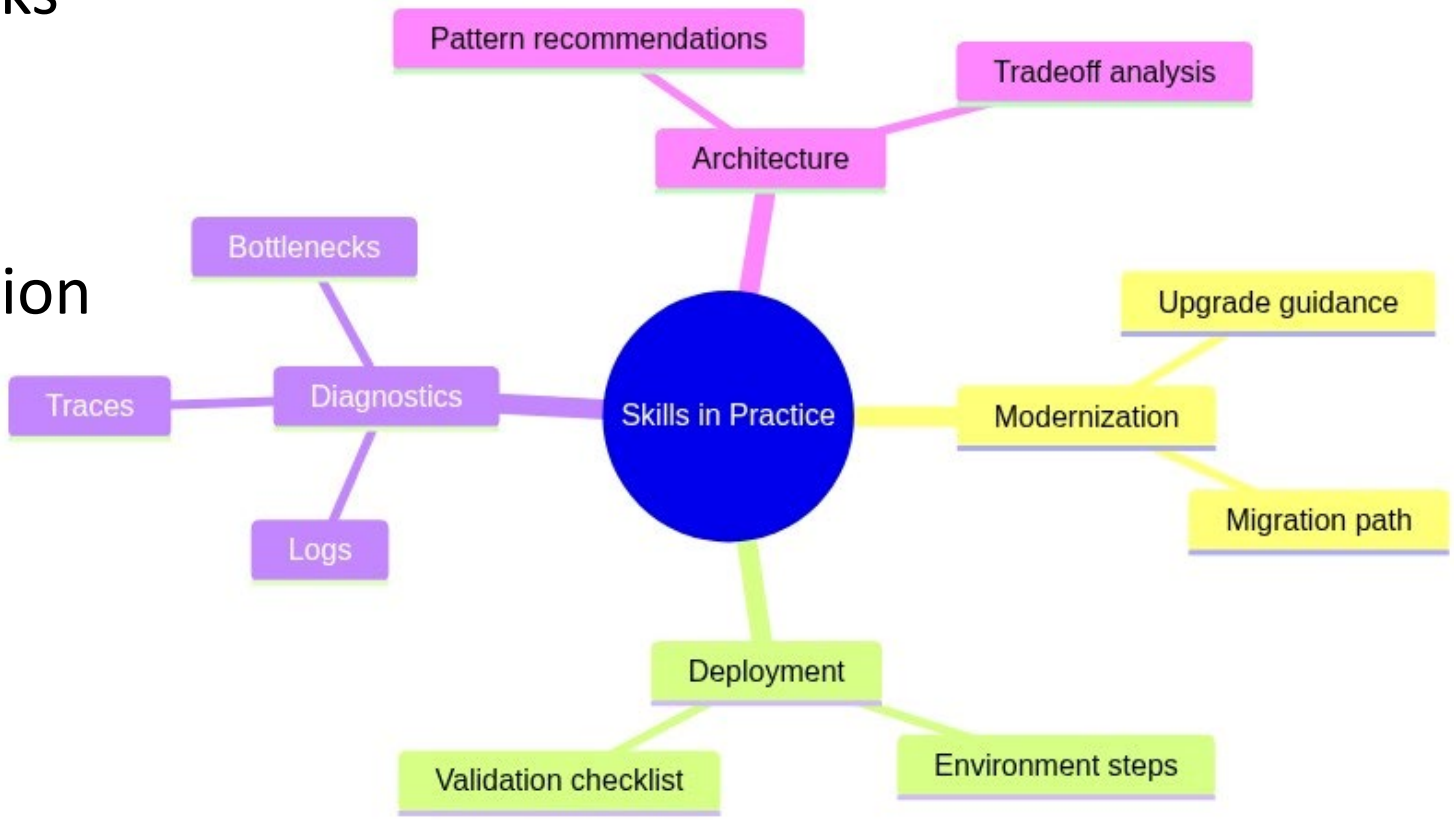
- Stable repo instructions reduce repeated context you would otherwise restate in every prompt.
- Reusable prompt/agent files shorten ad-hoc prompts and reduce redundant chat history.
- Better grounding usually means fewer retries, which lowers total token consumption per task.

Migrating from older agents.md

- Split old generic guidance into:
  - repo-wide defaults in copilot-instructions.md
  - task-specific agents in .github/agents/
- Keep instructions short, explicit, and repository-specific.

Skill usage examples

- App modernization scenarios
- Cloud deployment playbooks
- Advanced diagnostics workflows
- Architecture recommendation workflows



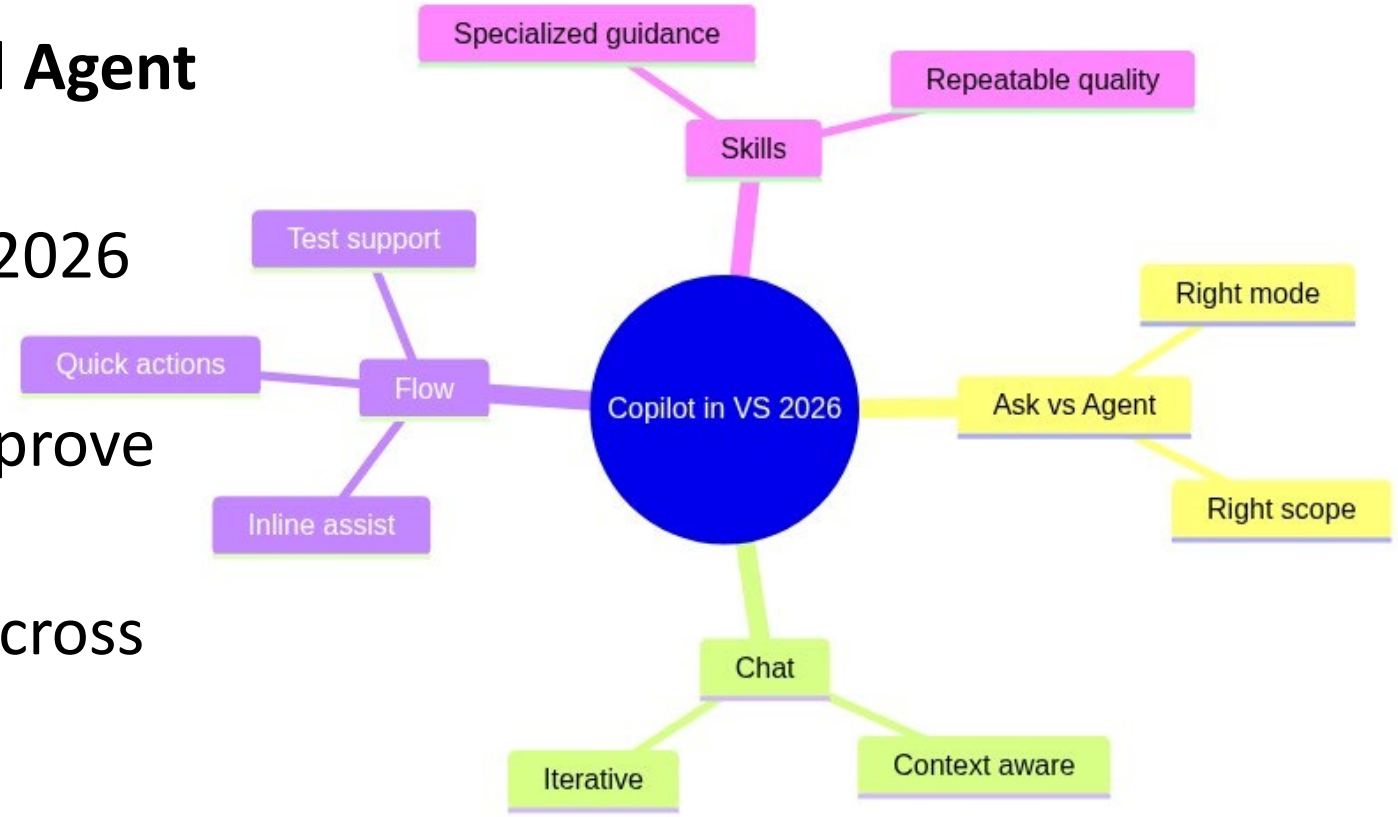
Skill demos

Modernize this Web Forms application to Blazor with ASP.NET Core 10. Keep the existing functionality, reuse as much code as possible, and follow best practices for the new framework.

Launch the profiler to start the Aspire AppHost. From there, trigger the bot API to play multiple games. Analyze the memory usage.

Summary

- Use **Ask** for fast guidance and **Agent** for multi-step execution
- Copilot Chat in Visual Studio 2026 shortens feedback loops
- AI features in normal flow improve daily productivity
- Skills help scale consistency across teams



Resources

- GitHub Copilot documentation
- GitHub Copilot in Visual Studio
- Prompt engineering with GitHub Copilot
- Microsoft Dev Blogs
- CN innovation
- My Blog

Q & A

Common questions:

- *Will Copilot replace developers?* — No. It amplifies developers; judgment stays human.
- *Should we always use agent mode?* — No. Use it when task scope is multi-step.
- *How do we keep output reliable?* — Provide constraints and validate with tests.
- *Can teams standardize usage?* — Yes, with prompt patterns and skills.

Thank You!

Enjoy Thrive Conference!

> *Build faster with AI, ship safer with engineering discipline.*

Thrive Conference 2026
Slides and samples: **github.com/CNILearn/thrive2026**

THRIVE CONFERENCE

Thank you

We greatly value your input. Please take a moment to complete our survey, accessible either through the Run.events app or by the link below:

https://bit.ly/Thrive26Survey

LightningTools Pearson span Leading Learning Partners Association FALA VSE RASTE! Cockta DoNatural ARGETA barcaffè CONDITOS

Lisjak Fino s Kristino Hydro Flask We Make It. You Own It. Računalniške novice IVARNOST.SI

THRIVE CONFERENCE

See you all next year